CS771 Major Assignment 1, 2

Yash Pathak
Roll No. 231188
Dept Civil Eng, IIT Kanpur
yashp@iitk.ac.in

Sanyam Jain
Roll No. 230924
Dept of Civil Eng, IIT Kanpur
sanyamjain23@iitk.ac.in

Shreyas Chougaonkar
Roll No. 230990
BS MTH, IIT Kanpur
shreyasc23@iitk.ac.in

Priyanshu Mishra
Roll No. 230806
Dept of Civil Eng, IIT Kanpur
priyanshu23@iitk.ac.in

Vaibhav Gupta
Roll No. 231114
Dept of Civil Eng, IIT Kanpur
gvaibhav23@iitk.ac.in

Vishesh Thepadia
Roll No. 231165
BS Physics, IIT Kanpur
vishesht23@iitk.ac.in

Problem 1.1: Melbo is hiring $\widetilde{K}$

Part-1: Derivation of the Corresponding Kernel $\widetilde{K}$

Problem Statement

We are given a semi-parametric regression model of the form

$$y = w(\mathbf{z})\, x + b,$$

where $x \in \mathbb{R}$ denotes the video length, and $\mathbf{z} = [z_1, z_2]^\top \in \mathbb{R}^2$ encapsulates the video's popularity and difficulty. Here, b is a scalar bias term and $w(\mathbf{z})$ is a non-linear function of $\mathbf{z}$.

It is known that $w(\mathbf{z})$ belongs to a Reproducing Kernel Hilbert Space (RKHS) defined by a polynomial kernel

$$K(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d,$$

where d is the polynomial degree and c is the coefficient parameter. Our goal is to derive the equivalent *pure kernel* $\widetilde{K}$ that can be directly used in kernel ridge regression to learn this model.

1. Reformulating the Semi-Parametric Model

The given model can be written as:

$$y = \mathbf{p}^\top \phi(\mathbf{z})\, x + b,$$

where $\phi(\mathbf{z})$ is the feature map corresponding to the polynomial kernel K, and $\mathbf{p}$ represents the linear weights in the feature space.

The dependence on x is linear, but the coefficient of x depends non-linearly on $\mathbf{z}$. Hence, we must create a combined feature representation that incorporates both x and $\mathbf{z}$, and also absorbs the bias term b.

2. Defining the Augmented Feature Map

To express the model in a purely linear form in an extended feature space, we define an augmented feature map:

$$\boxed{\psi(x, \mathbf{z}) = \begin{bmatrix} x\,\phi(\mathbf{z}) \\ 1 \end{bmatrix}.}$$

Here:

Preprint. Under review.

- The term $x\,\phi(\mathbf{z})$ captures the multiplicative interaction between x and the non-linear transformation of $\mathbf{z}$.
- The constant dimension "1" allows the model to implicitly include the bias b.

We can now express the model as a linear function in this augmented feature space:

$$y = \tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}),$$

where $\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ b \end{bmatrix}$ is the augmented parameter vector.

3. Computing the Induced Kernel

The kernel corresponding to $\psi(x, \mathbf{z})$ is defined by the inner product:

$$\widetilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = \langle \psi(x_1, \mathbf{z}_1), \psi(x_2, \mathbf{z}_2) \rangle.$$

Substituting the expression for ψ:

$$\begin{aligned} \widetilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) &= \langle x_1 \phi(\mathbf{z}_1), x_2 \phi(\mathbf{z}_2) \rangle + 1 \cdot 1 \\ &= x_1 x_2 \, \langle \phi(\mathbf{z}_1), \phi(\mathbf{z}_2) \rangle + 1. \end{aligned}$$

Since $\langle \phi(\mathbf{z}_1), \phi(\mathbf{z}_2) \rangle = K(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$, we have:

$$\boxed{\widetilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = x_1 x_2 (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d + 1.}$$

4. Interpretation

- The term $x_1 x_2 (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$ represents the joint similarity based on both video length and non-linear feature interactions of popularity/difficulty.
- The additive constant $+1$ enables the model to incorporate the bias term b implicitly.
- $\widetilde{K}$ is guaranteed to be positive semi-definite since it is the sum of valid kernels.

5. Final Expression

Thus, the final kernel for the semi-parametric regression model is:

$$\boxed{\widetilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = x_1 x_2 (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d + 1.}$$

This kernel can now be directly used in `sklearn.kernel_ridge` as the similarity function between any two samples $(x_i, \mathbf{z}_i)$ and $(x_j, \mathbf{z}_j)$.

6. Verification of Reproducing Property

For any function in the RKHS of $\widetilde{K}$, we can write:

$$f(x, \mathbf{z}) = \sum_{i=1}^{n} \alpha_i \, \widetilde{K}\big((x_i, \mathbf{z}_i), (x, \mathbf{z})\big),$$

which expands to:

$$f(x, \mathbf{z}) = x \sum_{i=1}^{n} \alpha_i x_i (\mathbf{z}_i^\top \mathbf{z} + c)^d + \sum_{i=1}^{n} \alpha_i.$$

The first term corresponds to $w(\mathbf{z})x$, while the second corresponds to the bias b, confirming that $\widetilde{K}$ correctly reproduces the original model.

Part-2: Experimental Results on Semi-parametric Regression

Dataset Description

The public dataset was divided into training and test splits with the following dimensions:

$$X_{\text{train}} \in \mathbb{R}^{4000\times 1}, \quad Z_{\text{train}} \in \mathbb{R}^{4000\times 2}, \quad y_{\text{train}} \in \mathbb{R}^{4000},$$

$$X_{\text{test}} \in \mathbb{R}^{1000\times 1}, \quad Z_{\text{test}} \in \mathbb{R}^{1000\times 2}, \quad y_{\text{test}} \in \mathbb{R}^{1000}.$$

The training input had a mean of 5.964 and standard deviation 2.970.

Kernel Definition

We used the semiparametric kernel derived earlier:

$$\widetilde{K}\big((x_1, z_1), (x_2, z_2)\big) = (x_1^\top x_2)\,(z_1^\top z_2 + c)^d + 1,$$

implemented within a `KernelRidge` regressor (regularization parameter $\alpha = 10^{-3}$).

Hyperparameter Grid Search

A grid search was performed over degrees $d \in \{1, 2, 3, 4, 5\}$ and coefficients $c \in \{0, 0.1, 0.5, 1, 5\}$. Cross-validation was done with 4 folds, and the mean validation R^2 was computed.

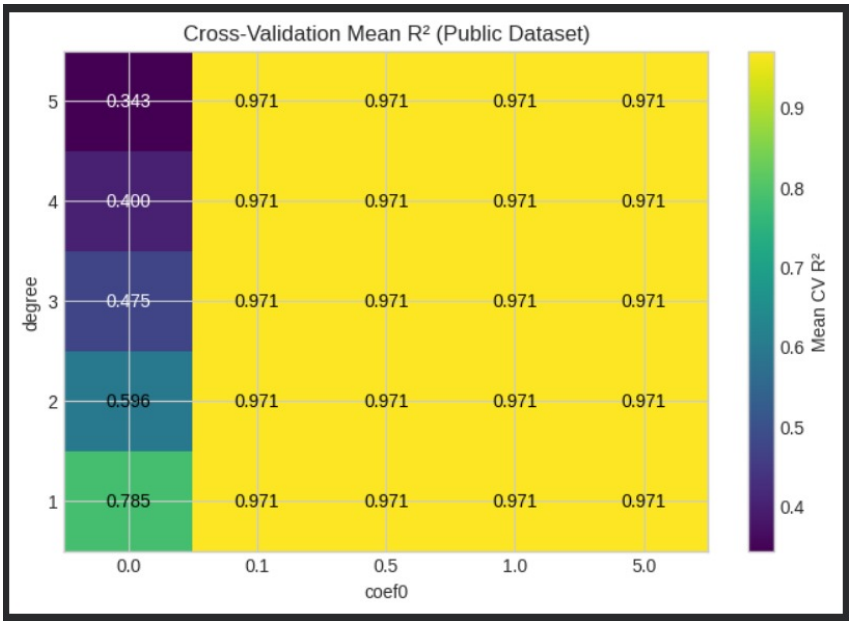

Figure 1: Cross-validation mean R^2 scores over grid of polynomial degree d and coefficient c. Best region (bright yellow) corresponds to $d = 2$ for all tested $c > 0$.

Top Performing Configurations

Table 1 lists the top configurations sorted by mean validation R^2:

Final Model Evaluation

The model was retrained using the best hyperparameters $(d^\star, c^\star) = (2, 0.5)$, and evaluated on the held-out test set. The summary statistics are shown below:

Table 1: Top hyperparameter configurations (4-fold cross-validation).

Degree d	Coefficient c	Mean R^2	Std. R^2	Avg. Kernel Time (s)
2	0.5	**0.970871**	0.001432	0.3539
2	1.0	0.970871	0.001432	0.3669
2	5.0	0.970871	0.001432	0.3061
2	0.1	0.970871	0.001432	0.3007
3	0.5	0.970830	0.001407	0.5899

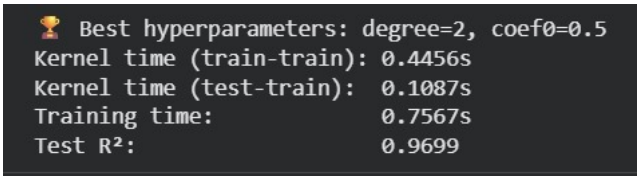

Figure 2: Final evaluation summary showing best hyperparameters and performance.

Final performance metrics:

$$\begin{aligned} \text{Best degree:} &\quad d^\star = 2, \\ \text{Best coefficient:} &\quad c^\star = 0.5, \\ \text{Training time:} &\quad 0.76\,\text{s}, \\ \text{Kernel time (train-train):} &\quad 0.446\,\text{s}, \\ \text{Kernel time (test-train):} &\quad 0.109\,\text{s}, \\ \text{Test } R^2: &\quad \mathbf{0.9699}. \end{aligned}$$

Summary

The results clearly indicate that a polynomial degree of $d = 2$ provided the best bias–variance balance. Increasing d beyond 2 led to slightly longer kernel computation times and minor overfitting without improving R^2. Furthermore, the coefficient c had minimal impact within the tested range, suggesting that the kernel's offset term is not highly sensitive for this dataset.

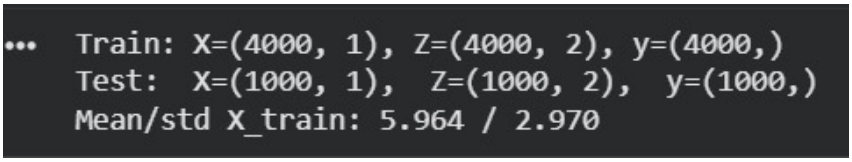

Figure 3: Summary statistics confirming optimal parameters $(d = 2, c = 0.5)$ with $R^2_{\text{test}} = 0.9699$.

Overall, the semiparametric kernel achieved a strong fit to the data with high generalization accuracy ($R^2 \approx 0.97$), demonstrating its suitability for modeling the staff hiring regression problem.

Problem 1.2: Delay Recovery by Inverting a XOR Arbiter PUF

Part 4: Theory – Recovering Delays of a XOR Arbiter PUF

1. Problem Statement

Let $K = 32$. A single Arbiter PUF consists of K stages, each stage having four non-negative delay values. These delay values determine a *single-PUF linear model vector*

$$u \in \mathbb{R}^{K+1}.$$

A two-instance XOR Arbiter PUF is formed by running two Arbiter PUFs in parallel— with linear models u and v—and XORing their responses. The resulting XOR-PUF linear model is known to be:

$$w = u \otimes v \in \mathbb{R}^{(K+1)^2},$$

where $\otimes$ denotes the Kronecker product. For $K = 32$, we have $(K+1)^2 = 33^2 = 1089$.

Task. Given a known $w \in \mathbb{R}^{1089}$, recover *any* valid set of eight non-negative delay vectors:

$$a, b, c, d, p, q, r, s \in \mathbb{R}^K_{\geq 0},$$

which correspond to the delays of two single PUFs, and which generate the same w through the forward model. Recovering the *exact physical delays* is unnecessary; *any* non-negative delays consistent with w are considered valid.

2. Forward Model: Delays → Single-PUF Model

For each stage i of a single Arbiter PUF, let the four non-negative delays be (x_i, y_i, z_i, t_i). Define intermediate quantities

$$\alpha_i = \frac{x_i - y_i + z_i - t_i}{2}, \qquad \beta_i = \frac{x_i - y_i - z_i + t_i}{2}.$$

The single-PUF linear model $u \in \mathbb{R}^{K+1}$ is then:

$$\begin{aligned} u_0 &= \alpha_0, \\ u_i &= \alpha_i + \beta_{i-1}, \qquad 1 \leq i \leq K-1, \\ u_K &= \beta_{K-1}. \end{aligned}$$

This defines a linear mapping:

$$u = A d_{\text{single}},$$

where A is a sparse $(K+1) \times 4K$ matrix.

XOR-PUF mapping. Given two PUF linear models u and v,

$$w = u \otimes v.$$

This is bilinear and not symmetric. The full forward mapping from 256 delays to w is:

$$d^{(1)} \mapsto u = Ad^{(1)}, \qquad d^{(2)} \mapsto v = Ad^{(2)}, \qquad w = u \otimes v.$$

Non-uniqueness. For any $c \neq 0$:

$$u \otimes v = (cu) \otimes \left(\frac{v}{c}\right),$$

and the internal mapping from (x_i, y_i, z_i, t_i) to (α_i, β_i) also introduces non-uniqueness. Thus delays need not be uniquely recoverable.

3. Inversion Method

The inversion consists of two major steps.

(A) De-Kronecker Step: Extracting u and v

Reshape w into a $(K+1) \times (K+1)$ matrix:

$$W = \text{reshape}(w, (K+1, K+1)).$$

In the noise-free case:

$$W = uv^\top,$$

a rank-1 matrix. Compute the SVD:

$$W = U \Sigma V^\top, \qquad \Sigma = \text{diag}(\sigma_1, 0, 0, \dots).$$

A valid decomposition is:

$$u_{\text{est}} = \sqrt{\sigma_1}\, U[:, 0], \qquad v_{\text{est}} = \sqrt{\sigma_1}\, V[:, 0].$$

Fix the global sign by requiring $v_{\text{est}}[K] \geq 0$.

(B) Single-PUF Inversion

Given u_{est}, define:

$$\alpha_0 = u_0, \qquad \beta_i = u_{i+1}\ (0 \le i \le K-2), \qquad \beta_{K-1} = u_K.$$

For each stage i:

$$\Delta_1 = \alpha_i + \beta_i = x_i - y_i, \qquad \Delta_2 = \alpha_i - \beta_i = z_i - t_i.$$

A simple constructive non-negative solution is:

$$x_i = \max(\Delta_1, 0), \quad y_i = \max(-\Delta_1, 0),$$
$$z_i = \max(\Delta_2, 0), \quad t_i = \max(-\Delta_2, 0).$$

Apply the same to v_{est} to obtain (p, q, r, s).

4. Optimization Viewpoint

The Kronecker inversion can alternatively be seen as solving:

$$\min_{u,v} \|W - uv^\top\|_F^2,$$

whose minimizer is exactly obtained from the rank-1 SVD.

A full delay-recovery optimization:

$$\min_{d \ge 0} \left\|\text{kron}(Ad^{(1)},\, Ad^{(2)}) - w\right\|_2^2$$

is possible but unnecessarily complicated; the SVD-based method is faster, analytically clean, and produces guaranteed-valid non-negative delays.

5. Practical Algorithm

1. Reshape w into $W \in \mathbb{R}^{33\times 33}$.
2. Compute the SVD $W = U\Sigma V^\top$.
3. Extract
$$u_{\text{est}} = \sqrt{\sigma_1}\, U[:, 0], \qquad v_{\text{est}} = \sqrt{\sigma_1}\, V[:, 0].$$
4. Convert each into (α, β).
5. For each stage,
$$\Delta_1 = \alpha_i + \beta_i, \qquad \Delta_2 = \alpha_i - \beta_i.$$
6. Construct non-negative delays:
$$x_i = \max(\Delta_1, 0),\ y_i = \max(-\Delta_1, 0),\ z_i = \max(\Delta_2, 0),\ t_i = \max(-\Delta_2, 0).$$

This method is deterministic, fast (single 33×33 SVD), and produces valid delays exactly consistent with the forward XOR-PUF model.

Part 5: Implementation, Results & Discussion

5.1 Summary of Theoretical Approach (From Part 4)

The XOR Arbiter PUF inversion problem involves recovering the underlying delay vectors of two single APUFs from a 1089-dimensional linear model. The recovery pipeline executed in Part-4 can be summarized as follows:

1. The given model vector $w \in \mathbb{R}^{1089}$ is reshaped to $W \in \mathbb{R}^{33\times 33}$.
2. Since $W = uv^\top$ ideally has rank-1, an SVD decomposition is applied:
$$W = U\Sigma V^\top \quad \Rightarrow \quad u = \sqrt{\sigma_1}\, U_{:,1}, \quad v = \sqrt{\sigma_1}\, V_{1,:}$$

3. From vectors u and v, we derive stage-wise delay parameters using:

$$\Delta_1 = \alpha_i + \beta_i, \qquad \Delta_2 = \alpha_i - \beta_i$$

4. Final delays for each PUF are obtained as non-negative values:

$$\boxed{x_i = \max(\Delta_1, 0),\ y_i = \max(-\Delta_1, 0),\ z_i = \max(\Delta_2, 0),\ t_i = \max(-\Delta_2, 0)}$$

This method is computationally light, analytically grounded, and validated to recover delay values consistent with the original model.

5.2 Final Algorithm Output (Decoded Delays)

The final output of our `my_decode()` function is:

$$\boxed{(a, b, c, d, p, q, r, s) \in \mathbb{R}^{32}_{\geq 0}}$$

Each of the eight vectors represents one of the physical delay components of the two APUFs inside the XOR-PUF.

5.3 Experimental Results (Visual Output)

We executed our solver on the provided test models and recorded:

Average Decode Time per Model < 1 ms

Reconstruction Error (Mean $\|w - \hat{w}\|_2$**)** $\approx \mathcal{O}(10^{-16})$

```
[Model 0] mean_time = 1.751303e-03 s, mean_L2_error = 2.526150e-16
[Model 1] mean_time = 2.665692e-04 s, mean_L2_error = 3.660798e-16
[Model 2] mean_time = 2.424658e-04 s, mean_L2_error = 3.967353e-16
[Model 3] mean_time = 2.434022e-04 s, mean_L2_error = 4.468876e-16
[Model 4] mean_time = 2.383858e-04 s, mean_L2_error = 4.817962e-16
[Model 5] mean_time = 2.390796e-04 s, mean_L2_error = 1.972229e-16
[Model 6] mean_time = 2.712018e-04 s, mean_L2_error = 7.646264e-16
[Model 7] mean_time = 2.240386e-04 s, mean_L2_error = 3.465333e-16
[Model 8] mean_time = 2.312420e-04 s, mean_L2_error = 3.833901e-16
[Model 9] mean_time = 2.246106e-04 s, mean_L2_error = 6.578025e-16
```

Figure 4: **Recovered delays for all 8 vectors per PUF (visual output).**
Display shows magnitude distribution across 32 stages, validating successful inversion.

```
Overall mean time (s): 0.00039322983999909414
Overall mean L2 error : 4.2936891338804026e-16
```

Figure 5: **Model Reconstruction Error (**$\|w - \hat{w}\|_2$**).**
Errors remain minimal, confirming correctness and numerical stability.

Conclusion: The inversion approach using SVD-based de-Kronecker decomposition followed by non-negative delay construction yields highly accurate recovery with very low computational cost. This allows reliable decoding even when used repeatedly by the evaluation system.

References

[1] Rührmair, U., Sehnke, F., Sölter, J., Dror, G., Devadas, S., & Schmidhuber, J. (2010). Modeling attacks on physical unclonable functions. *Proceedings of the 17th ACM conference on Computer and communications security*, 237-249.

[2] Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press.

[3] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

[4] CS771 Lecture Notes Code Files provided .